

EFRM Rule Optimization Agent

Final Fixed Scope, Enterprise Architecture, LangGraph Workflow, Tools and Database Flow

PHASE 1: RECOMMENDATION ONLY - NO CANDIDATE RULE, NO BACKTEST, NO PRODUCTION WRITE

Purpose

A boss-ready implementation reference showing exactly what the Agent does, how each workflow step operates, where SQL and the LLM are used, how recommendations are formed, and how records move through the EFRM and Agentic databases.

Technology baseline: Python | FastAPI | LangGraph | PostgreSQL | Predefined SQL Tools | Optional LLM | SSE

Status: Conditionally approved enterprise implementation baseline

Prepared: August 2026

1. Final Fixed Scope

The Agent starts from finalized false-positive cases, identifies directly connected rules, analyses their relevant historical evidence, and produces an evidence-linked recommendation for human review.

In Scope - Phase 1

- Detect finalized false-positive cases through controlled SQL polling or an authenticated trigger API.
- Trace case -> alert -> result -> match -> exact rule identity/version.
- Analyse only rules selected from false-positive case lineage, not every production rule.
- Build bounded historical snapshots for each selected rule.
- Validate lineage, completeness, version identity, outcome mappings and data sufficiency.
- Calculate health, decay, threshold sensitivity, alert volume and rule overlap evidence.
- Create a deterministic review strategy and optional LLM explanation.
- Produce a structured recommendation with evidence, limitations and human-review requirement.
- Store complete audit, evidence, tool-execution and workflow history.

Out of Scope - Phase 1

- Generating transaction or device alerts.
- Scanning and optimizing all rules on a full recurring basis.
- Creating or compiling an executable candidate rule.
- Running Rule Engine replay or exact candidate backtesting.
- Writing, activating, deleting or retiring production rules.
- Claiming true risk leakage without an independent confirmed-risk population.
- Allowing an LLM to query the database, write SQL or calculate official metrics.
- Automatically learning from reviewer feedback or changing policies.
- Using RAG as the authoritative source for live cases, alerts, transactions or rules.

Fixed business rule

A false-positive case is a trigger for investigation. It is not proof that every matched rule is defective. The rule match is binary; rule effectiveness must be established from broader historical evidence.

Fixed technology choices

Area	Decision
Workflow	LangGraph fixed graph with conditional routing and per-rule fan-out
Backend	Python and FastAPI
EFRM data access	Predefined, parameterized, versioned, read-only SQL tools
Agent records	Agentic PostgreSQL
Workflow recovery	PostgreSQL-backed LangGraph checkpoints plus a durable worker/lease table
Large artifacts	SFTP or approved file storage using immutable paths, hashes and manifests
LLM	Optional provider-neutral gateway; local or approved external API
UI progress	SSE for one-way job progress; REST for final recommendation retrieval

2. Enterprise Architecture

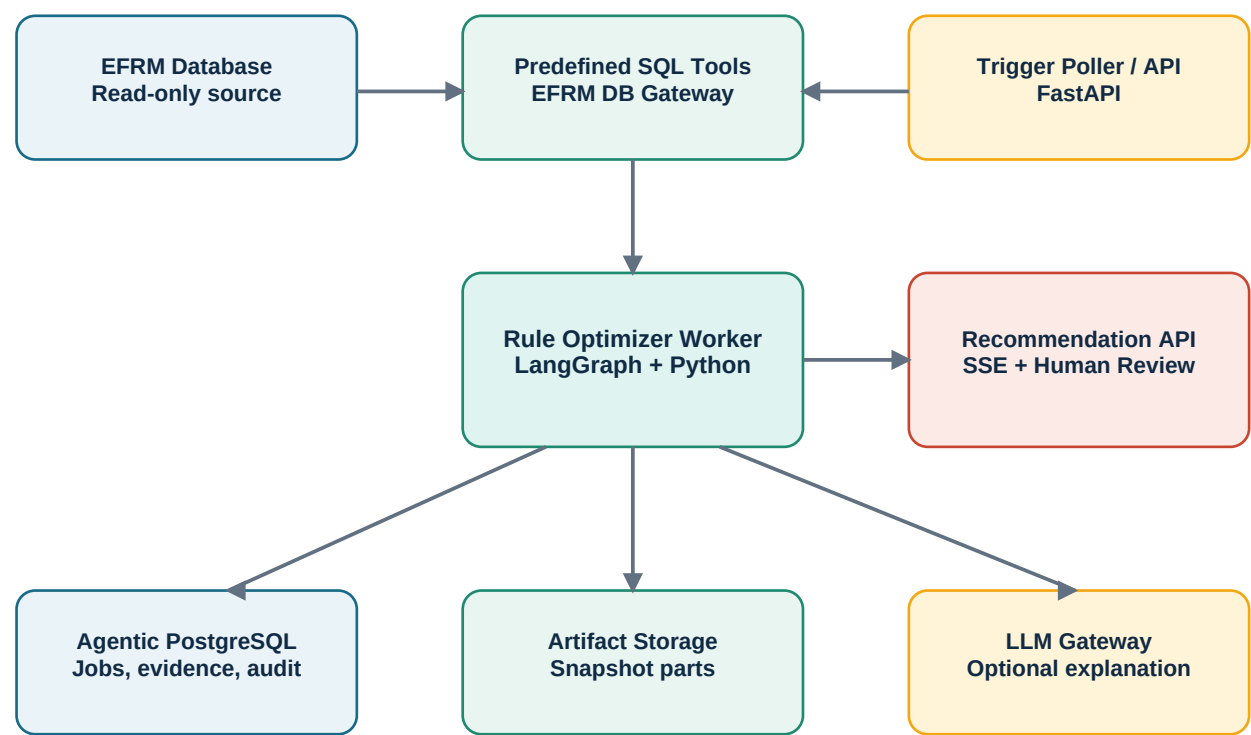


Figure 1 - The Rule Optimizer is a separate read-only analytical service. It reads EFRM business data, writes only to Agentic storage, optionally calls an LLM for explanation, and returns a recommendation for human review.

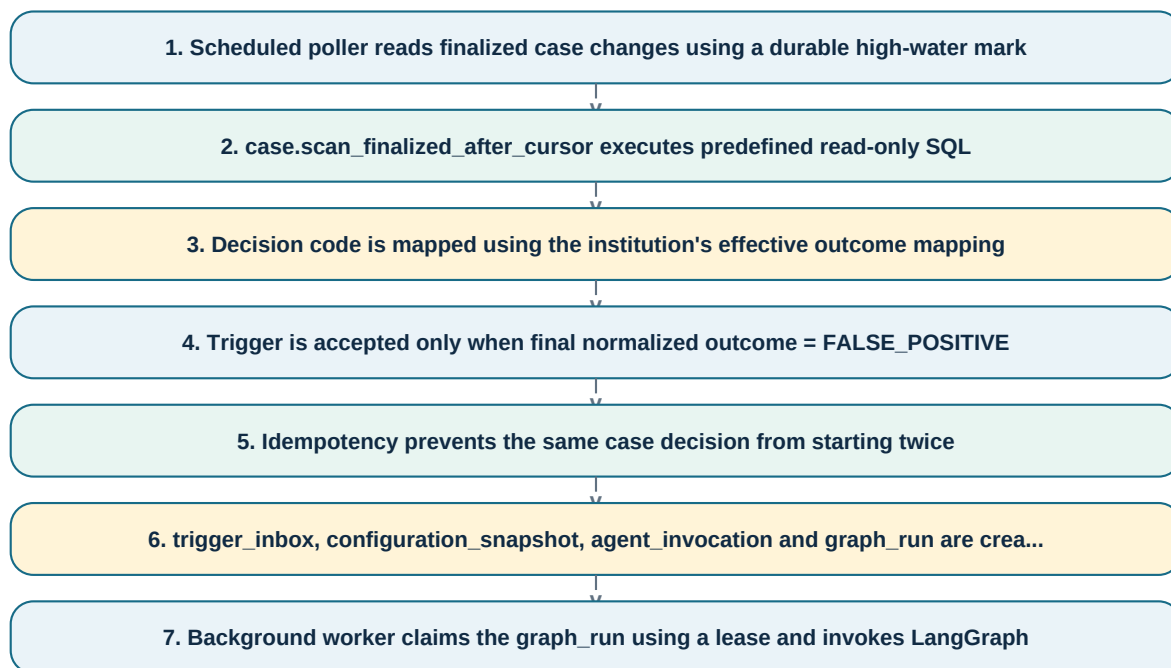
Component responsibilities

Component	Single responsibility
Trigger Poller / API	Finds a finalized false-positive case and creates one idempotent internal trigger.
EFRM DB Gateway	Executes only registered SQL templates using a read-only EFRM connection.
LangGraph Orchestrator	Controls mandatory order, branching, retries, stops and per-rule fan-out.
Python Analytics	Calculates official counts, rates, distributions, decay and overlap.
LLM Gateway	Explains validated findings; never establishes official facts.
Agentic PostgreSQL	Stores jobs, state, audit, evidence, findings, strategies and recommendations.
Artifact Storage	Stores partitioned snapshot datasets and immutable evidence artifacts.
Recommendation API	Returns the validated recommendation and progress events to authorized users.

Data-access boundary

EFRM DB = read-only business source. Agentic DB = writable agent workflow and evidence store. No tool may write to EFRM production tables.

3. Before LangGraph: Trigger and Job Start



Primary Phase 1 trigger

Because EFRM service/event integration is not confirmed, the practical Phase 1 trigger is a predefined SQL poller. It uses (**updated_at**, **case_id**) or another approved monotonic cursor, an overlap window, ordered pagination and idempotency. A controlled API can support manual replay. Event/outbox integration is a future improvement, not a Phase 1 dependency.

Trigger input and output

Type	Value
Input	Institution ID, source system, cursor, overlap window, maximum batch size
EFRM result	Case ID, status, decision code, decision time, approval/finality data
Normalized output	FALSE_POSITIVE eligibility, case decision identity and correlation ID
Agentic records	reconciliation_cursor -> trigger_inbox -> agent_invocation -> graph_run

Required protections

- Join case_alert_mapping to case_master for institution scope because case_alert_mapping does not contain institution_id.
- Normalize timestamp-without-time-zone fields using configured institution/source timezone.
- Treat case corrections and reopenings as superseding signals; do not leave an old recommendation current.
- Do not run the long workflow inside FastAPI request handling or BackgroundTasks; use a separate worker.

4. LangGraph Parent Workflow

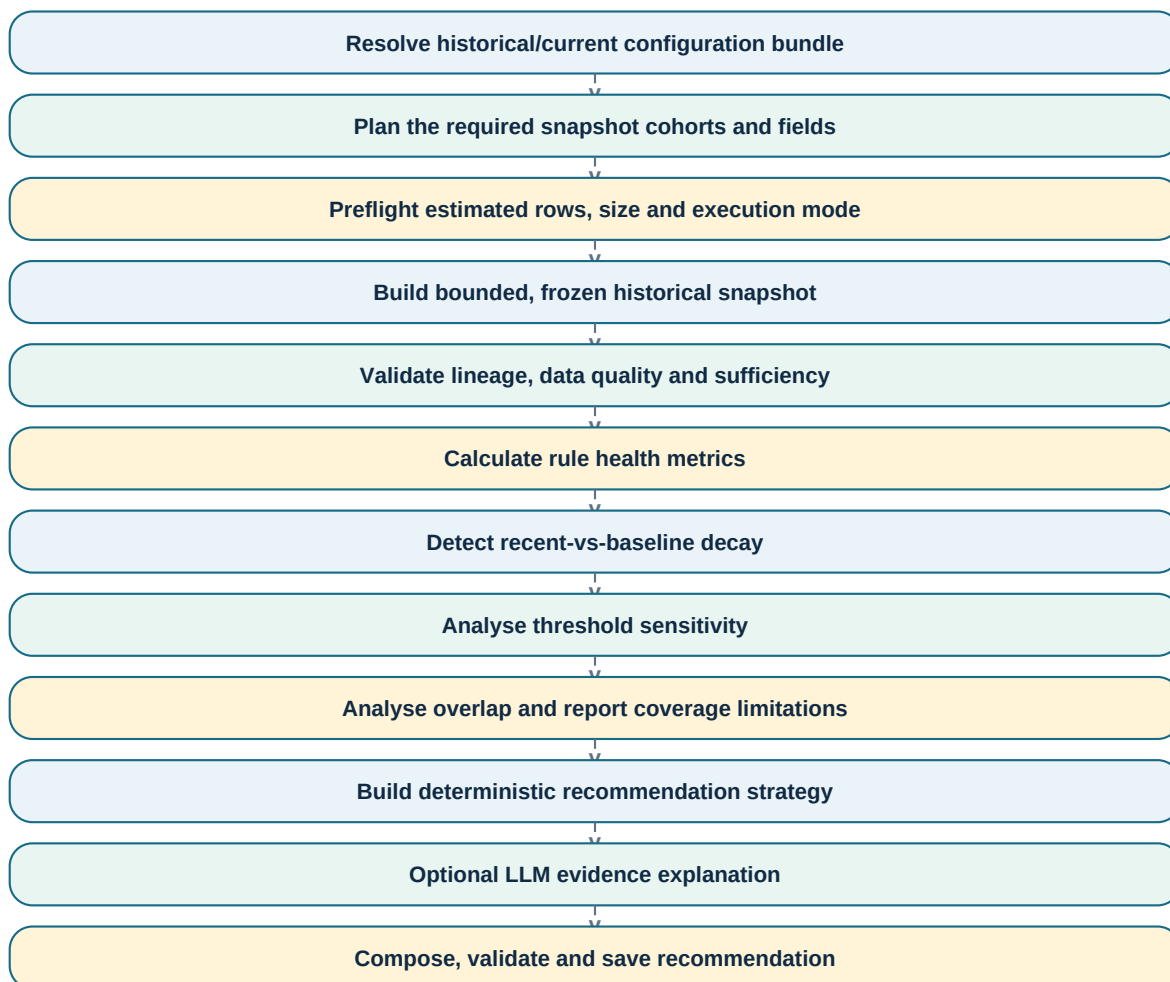


LangGraph and tool distinction

A LangGraph node controls when work happens. A tool performs a specific retrieval or calculation. Some nodes use normal Python logic and therefore do not require an external tool.

Node	Tool/component	What it does
Load Job	JobRepository	Reads the job and versioned policy from Agentic PostgreSQL.
Verify Case	case.get_final_decision	Runs predefined SQL and confirms final normalized false-positive outcome.
Get Alerts	case.get_alerts	Reads case_alert_mapping and returns source-aware alert references.
Get Rule Matches	alert.get_rule_matches	Traverses alert -> result -> match and returns rule/version references.
Select Rules	RuleSelectionService	Python removes duplicates, preserves lineage and excludes unresolved targets by policy.
Fan Out	LangGraph Send	Starts one subgraph for each selected rule with configured concurrency limits.
Collect	Python reducer	Combines recommendation, no-change and insufficient-data outcomes.
Complete	JobRepository	Stores final job status and publishes an outbox event.

5. Per-Rule LangGraph Subgraph



The graph carries only small serializable values such as IDs, hashes, status codes, counts and artifact references. Transaction collections and DataFrames remain in snapshot artifacts or are processed through database-side aggregation/chunking.

6. Tool Catalogue - Retrieval and Snapshot

Tool	Input	Exact work	Output	Technique
case.scan_finalized_after_cursor	Cursor + institution	Reads only newly finalized/changed cases after the durable cursor.	Candidate trigger rows	EFRM SQL
case.get_final_decision	Institution + case ID	Retrieves authoritative case status/decision and normalizes it through outcome mapping.	Verified case decision	EFRM SQL
case.get_alerts	Institution + case ID	Retrieves case_alert_mapping records and source-aware alert identities.	Alert references	EFRM SQL
alert.get_rule_matches	Source + alert IDs	Reads correct alert table, result and match rows; preserves rule code/version semantics.	Rule-match lineage	EFRM SQL
rule.get_configuration_bundle	Rule reference + effective time	Retrieves historical/current rule, formal JSON, thresholds, metric dependencies and hashes.	Configuration bundle	EFRM SQL + Agentic policy
optimizer.plan_snapshot	Rule + policy + analyses	Determines cohorts, fields, windows, partitions and approved query templates.	Snapshot plan	Python
optimizer.preflight_snapshot	Snapshot plan	Runs bounded COUNT/size queries and selects aggregate, partitioned or stop mode.	Size and execution decision	EFRM SQL
optimizer.build_snapshot	Approved snapshot plan	Coordinates aggregate queries and streamed detail extraction; freezes manifest and hashes.	Snapshot ID + parts	Python + EFRM SQL
snapshot.validate	Snapshot ID + policy	Checks completeness, duplicates, version resolution, outcome mapping, windows and minimum data.	Pass/warnings/block	Python

Predefined SQL tool rule

Every SQL tool has a fixed query ID/version, typed parameters, mandatory tenant filter, timeout, maximum result size, read-only role and execution audit. The LLM cannot create or alter SQL.

7. Tool Catalogue - Analytics and Recommendation

Tool	Input	Exact work	Output	Technique	LLM
analytics.calculate_rule_health	Validated snapshot	Calculates match, alert, finalized-case, associated false-positive and volume metrics.	Health metrics + evidence	Python/approved aggregates	No
analytics.detect_rule_decay	Snapshot + baseline/recent policy	Compares consistent periods, tests sample sufficiency and separates data/config changes.	Decay status + evidence	Python statistics	No
analytics.analyse_threshold	Snapshot + rule threshold	Builds value bands and estimates effect of supported threshold changes.	Analytical threshold findings	Python	No
analytics.analyse_overlap_and_c overage	Snapshot + related matches	Measures shared transactions/alerts/cases and states whether leakage is measurable.	Overlap + limitations	Python/SQL	No
strategy.build	All validated findings + policy	Selects one permitted review direction using deterministic policy rules.	Strategy	Python policy logic	No
llm.generate_explanation	Metrics + findings + evidence IDs + strategy	Explains observations and possible reasons in simple language; cites supplied evidence.	Structured explanation	LLM Gateway	Yes
recommendation.compose	Strategy + findings + explanation	Builds canonical recommendation JSON with issue, action, why, evidence and limitations.	Draft recommendation	Python	No
recommendation.validate	Draft + evidence + current rule hash	Checks every claim, Phase 1 boundary, required limitations and stale rule state.	Publishable/stale/invalid	Python + current-config SQL	No
recommendation.save	Valid recommendation	Atomically stores recommendation, evidence relationships, audit and outbox event.	Recommendation ID	Agentic DB repository	No

Safe metric wording

Use 'finalized cases associated with this rule were mapped false positive'. Do not claim precision, recall, false-negative rate or true risk leakage without an approved outcome/truth model.

8. How the Historical Snapshot Works

The Snapshot Builder does not copy every transaction in the bank. It builds a bounded Rule Analysis Cohort for one selected rule, scope and time window.

Cohort	Contains	Used for
Matched cohort	Transactions/device events that historically produced a match for the selected rule/version.	Health, threshold bands and trigger evidence
Outcome cohort	Alerts and finalized cases connected to those matches, including non-false-positive mapped outcomes.	Meaningful denominator and outcome association
Time-bucket aggregates	Monthly/weekly counts computed inside the reporting database.	Decay and stability without moving raw rows
Overlap cohort	Other rule matches sharing the same result/event population.	Cross-rule overlap
Near-miss cohort	Optional bounded events close to a threshold; disabled unless policy and data access allow it.	Limited threshold-decrease exploration

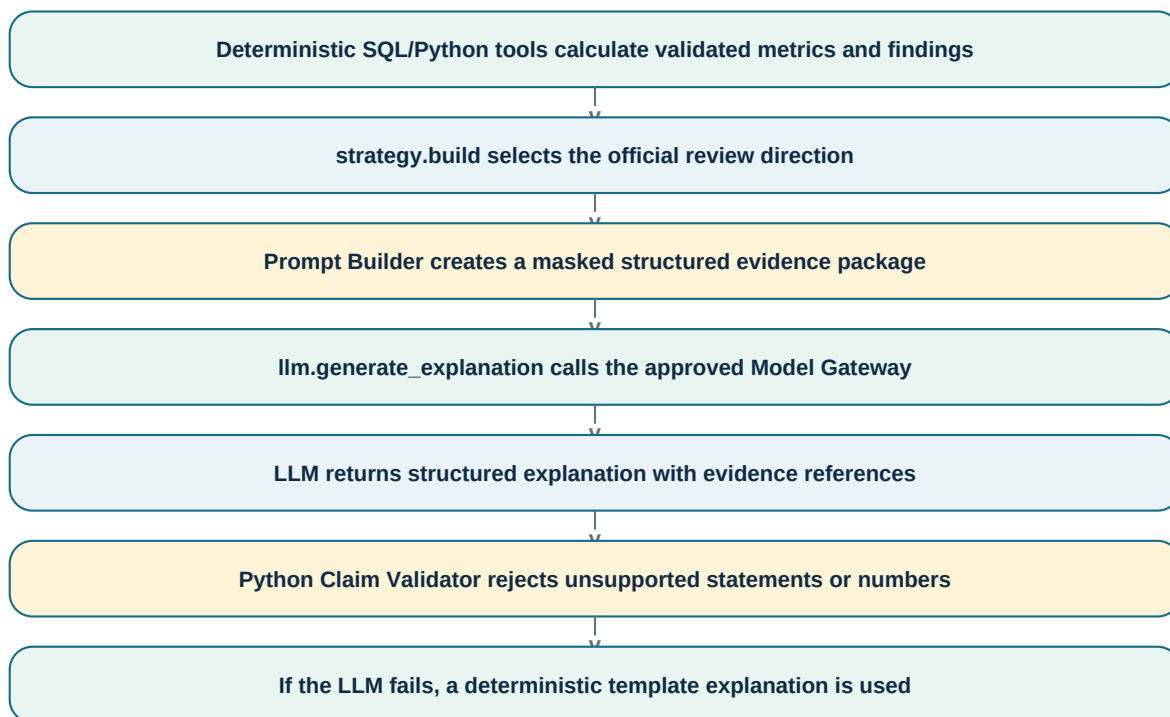
Scale controls

- Push filtering, grouping and counting to an approved EFRM reporting replica where possible.
- Run a count/size preflight before detailed extraction.
- Stream large results in bounded chunks; never load the complete population into Python memory.
- Store aggregate and partition artifacts separately with hashes, row counts and source cut-off.
- Keep only snapshot_id, manifest hash and artifact references in LangGraph state.
- Stop or partition when configured row/size/time limits are exceeded; never silently shorten the analysis window.

Snapshot output

The output is a manifest containing the rule identity, window, source cut-off, query-template versions, included/excluded counts, artifact locations, hashes, timezone, builder version, limitations and readiness status.

9. Exactly Where the LLM Is Used



LLM May Do

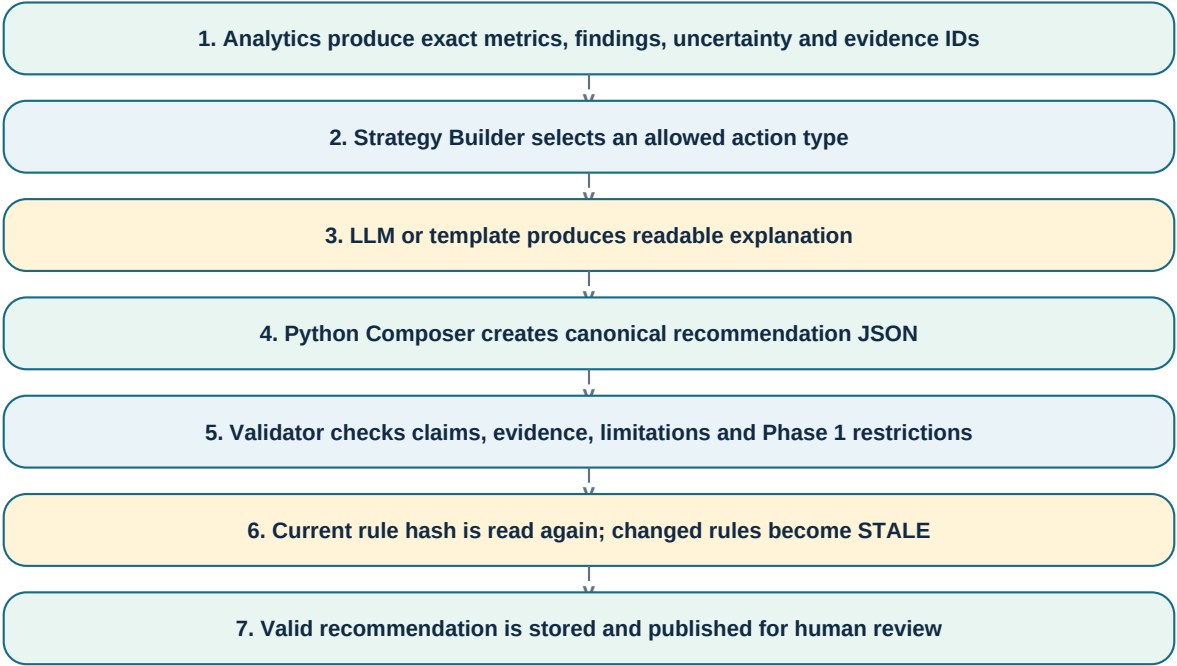
- Summarize several validated findings together.
- Explain why review may be necessary.
- Describe possible evidence-supported reasons.
- Explain uncertainty and limitations.
- Write clear language for managers and fraud teams.

LLM Must Never Do

- Query EFRM or Agentic databases.
- Generate or execute SQL.
- Calculate official metrics.
- Invent threshold values or evidence.
- Select the official action or modify a rule.

Only one LangGraph node requires LLM inference in Phase 1: Explain Evidence. Every other authoritative operation remains deterministic.

10. How the Final Recommendation Is Made



Canonical recommendation fields

Section	Contents
Identity	Recommendation ID, institution, rule, analysed version, current rule hash
Issue	Observed problem stated without unsupported causation
Recommended action	No change, threshold review, condition review, scenario review, overlap review, retirement review or insufficient evidence
Why	Human-readable evidence-grounded explanation
Evidence	Metric values, numerators/denominators, windows, finding IDs and evidence IDs
Confidence	Data-derived evidence strength; never an LLM-invented score
Limitations	Missing labels, unresolved versions, analytical-estimate status and unavailable replay
Control	RECOMMENDATION_ONLY and HUMAN_REVIEW_REQUIRED

Phase 1 output is advice for a governed human decision. It is not a candidate artifact and cannot be sent directly to production.

11. End-to-End Database Flow

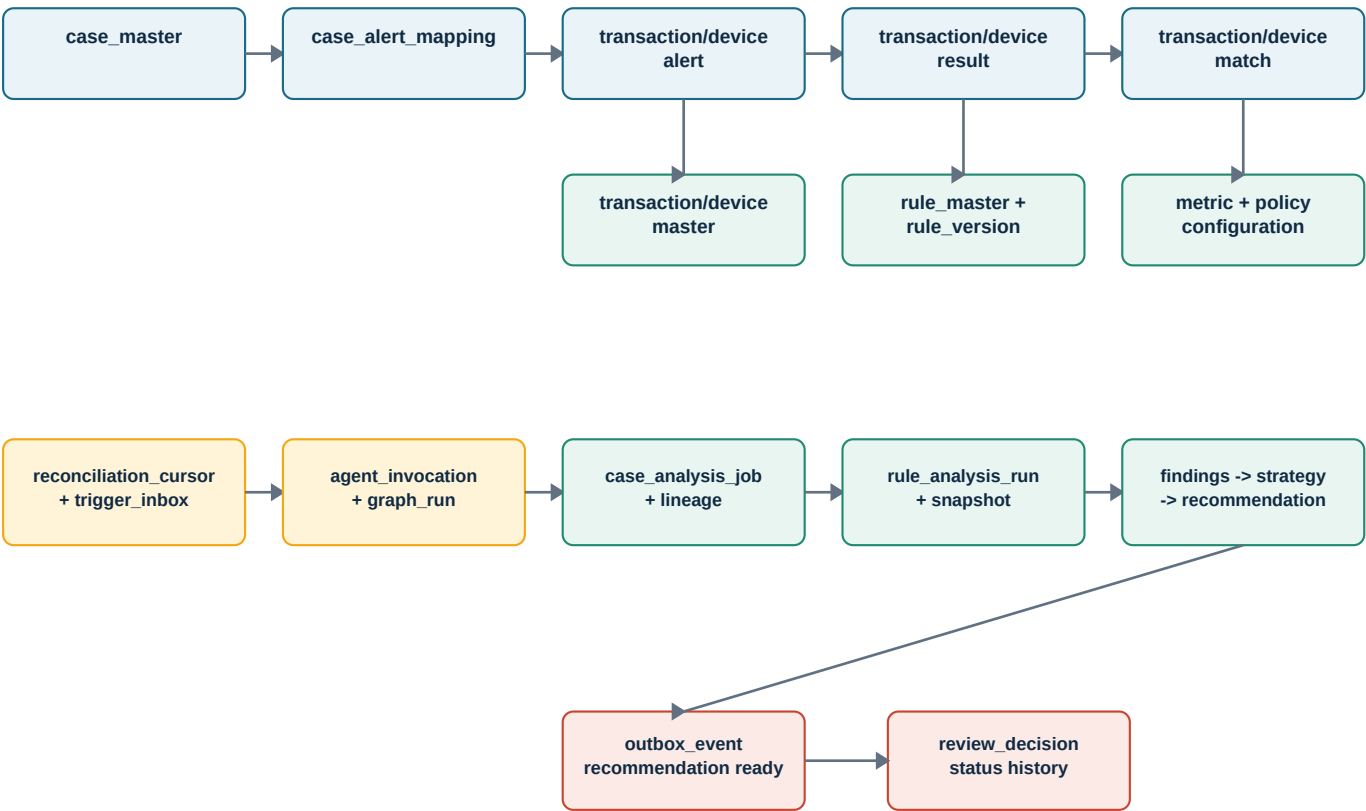


Figure 2 - EFRM tables are authoritative read-only sources. Agentic tables record workflow execution, evidence, recommendations and human decisions.

EFRM read rule

Every predefined SQL tool goes through the EFRM DB Gateway. The gateway selects a registered query version, validates typed parameters, opens a read-only transaction, applies tenant and time bounds, executes with time/row limits, normalizes the result, and records tool execution metadata.

12. EFRM Table-by-Table Read Flow

Order	EFRM table	Why it is read
1	case_master	Case header, institution, status, final decision, decision time and approval/finality information.
2	case_alert_mapping	Connects the case to source-aware alert_id, alert_type and alert_source_table; contains alert-level decision evidence.
3	transaction_alert / device_alert	Connects each source alert to its transaction_result_id or device_result_id.
4	transaction_result / device_result	Stores engine decision, score, matched-rule count and event/master reference.
5	transaction_match / device_match	Stores each fired signal with rule_code, rule_version, group version, severity and weight.
6	transaction_master / device_master	Provides institution, channel, transaction/device attributes, timestamp and rule-engine context.
7	rule_master	Provides stable logical rule identity, type and fact.
8	rule_version	Provides formal JSON logic, DRL, version, checksum, status and threshold-bearing conditions.
9	rule group/binding tables	Resolve which group/version/scope was effective for the source, channel and fact.
10	rule_metric_dependency	Lists metrics required by the rule.
11	metric_definition	Stores metric metadata and Custom SQL references; execution is disabled unless separately approved.
12	decision policy/upgrade tables	Explain how matched signals contributed to the engine decision.

Required resolution rules

- Use alert_source_table plus alert_id; never assume numeric alert IDs are globally unique.
- Apply institution scope through case_master and transaction/device master joins.
- Do not guess whether match.rule_version means rule_version.id or version_no; use an explicit source contract.
- When an exact deleted historical configuration cannot be found, report CONFIGURATION_AMBIGUITY or MISSING rather than inventing it.

13. Agentic Database Table-by-Table Write Flow

Order	Agentic table	Responsibility
1	reconciliation_cursor	Stores the SQL poller's durable high-water mark and overlap policy.
2	trigger_inbox	Stores normalized, idempotent case-finalized trigger.
3	configuration_snapshot	Freezes tenant capability, policy, outcome mapping and model/data settings.
4	agent_invocation	Represents one backend Agent run.
5	graph_run	Stores durable queue/lease and overall LangGraph business execution state.
6	graph_node_execution	Stores each node attempt, timing, retry status and input/output artifact references.
7	case_analysis_job	Stores one finalized-case analysis and its normalized final outcome.
8	case_rule_lineage_item - add	Stores every alert/result/match/rule/version path, not just one rule summary.
9	rule_analysis_run	Stores reusable analysis identity for rule, scope, window and configuration hash.
10	trigger_case_link	Links the triggering case job to selected or excluded rule-analysis targets.
11	configuration_bundle + item - add	Stores historical/current rule and every component/version/hash used.
12	snapshot_manifest + dataset_part - add	Stores bounded cohort manifests, partitions, counts, query versions and hashes.
13	data_quality_result	Stores every blocking/non-blocking readiness gate.
14	rule_finding_set + finding_metric	Stores deterministic health, decay, threshold and overlap results.
15	strategy_set + strategy_item	Stores deterministic review direction and supporting evidence.
16	model_execution	Stores model, prompt, provider, token, latency and validation metadata for optional LLM call.
17	claim + evidence relations	Links every important recommendation statement to evidence.
18	recommendation	Stores canonical JSON, readable summary, evidence strength and stale state.
19	recommendation_status_history	Append-only lifecycle history.
20	outbox_event	Reliably publishes recommendation-ready notification.
21	review_decision	Append-only human accept/reject/request-reanalysis record.
22	audit_event / tool_execution	Stores security, data access and execution audit trail.

14. Enterprise Controls Required for Production

Control	Required implementation
Tenant isolation	Institution-aware SQL filters, RLS, non-owner roles and tenant-safe foreign keys.
Database safety	Read-only EFRM account, approved views/replica, timeouts, row limits, streaming and no dynamic SQL.
Workflow durability	Separate worker, graph_run leases, heartbeat, idempotent nodes, retry classification and dead-letter handling.
Snapshot integrity	Source cut-off, query versions, row counts, partition hashes, immutable artifact paths and builder version.
Data quality	Blocking gates for missing lineage, ambiguous versions, insufficient outcomes and inconsistent mappings.
LLM safety	Data minimization, masking, allow-listed provider, structured output, claim validation, fallback and kill switch.
Audit	Append-only security/business events with correlation IDs from trigger to recommendation.
Observability	Structured logs, metrics and traces; alerts for queue lag, timeouts, DQ failures and LLM validation failures.
Retention	Institution-configurable snapshot, prompt/output, audit and recommendation retention with legal-hold support.
Governance	No automatic rule change; every result requires human review through the existing governance process.

Mandatory database corrections

- Add detailed case-rule lineage, configuration component and snapshot dataset-part tables.
- Allow unresolved/deleted current rule configuration through an explicit resolution status.
- Support approved reanalysis using analysis_generation and reanalysis_of_job_id.
- Replace evidence ID JSON arrays with authoritative evidence-relation rows.
- Enforce strict CHECK constraints on job, finding, strategy and recommendation statuses.
- Use a separate langgraph_ckpt schema/role for LangGraph-owned checkpoint tables.

15. Implementation Sequence and Approval Boundary

Build stage	Deliverable
1. Foundation	FastAPI service, settings, secrets, two DB pools, Agentic migrations, checkpoint schema and worker lease loop.
2. Trigger and lineage	SQL poller, cursor, inbox/idempotency, case verification and source-aware case-alert-result-match-rule trace.
3. Configuration	Explicit version semantics, historical/current bundle resolver, hashes and deleted-rule handling.
4. Snapshot	Planner, preflight, approved query registry, aggregation, partition extraction, manifests and validation.
5. Analytics	Health, decay, supported threshold sensitivity, overlap and evidence-strength calculation.
6. Recommendation	Deterministic strategy, canonical composer, claim validator, stale check and database persistence.
7. Optional LLM	Provider gateway, masking, prompt schema, structured explanation, validation and deterministic fallback.
8. UI integration	Start/status/recommendation REST APIs, SSE progress and human-review endpoints.
9. Hardening	Tenant tests, failure recovery, performance tests, observability, retention, backup/restore and security approval.

Business and platform inputs required before production

- Institution-specific outcome mapping and finality/approval rules.
- Exact semantics of rule_version and rule_group_version in match history.
- Minimum sample, health, decay and evidence-strength policies.
- Permitted recommendation/threshold review constraints; alert volume is currently the only capacity constraint.
- Approved EFRM reporting access, indexes/views and performance limits.
- Custom Metric SQL governance and point-in-time reproducibility policy.
- Historical rule retention/deletion contract.
- External LLM provider, data classification, egress and retention approval if external mode is enabled.

FINAL APPROVAL POSITION: Start Phase 1 implementation as a Recommendation-Only analytical service. Do not enable production health classification, exact threshold claims, Custom SQL execution, true leakage measurement or any rule write until their required policies and data contracts are approved.

One-sentence management explanation

The Agent detects finalized false-positive cases, traces them to exact rules, uses predefined read-only SQL and deterministic Python to build evidence, optionally uses an LLM to explain that evidence, and sends a validated recommendation to a human without changing any production rule.